

CHRIST (Deemed to be University)
Department of Computer Science
NEOSTATS sponsored INNOVATION SPRINT
06.06.2026 (8:00 am to 12:00 noon)

Hackathon Use Case Document

Intelligent Multi-Mode Conversational AI Agent

1. Overview

The objective of this hackathon is to build an Intelligent Conversational AI Agent capable of answering user questions through multiple knowledge sources while maintaining conversational context.

The solution should provide a seamless chat experience where the agent can intelligently decide whether to:

- Answer directly using its LLM knowledge.
- Search the web for real-time information.
- Retrieve information from documents using RAG (Retrieval-Augmented Generation).
- Use conversation memory to answer follow-up questions.

The chatbot must be implemented using a fully open-source technology stack and exposed through a Streamlit-based user interface.

2. Business Problem

Organizations frequently require a unified conversational interface that can:

- Answer general knowledge questions.
- Retrieve information from internal enterprise documents.
- Access current information available on the internet.
- Remember previous interactions to support multi-turn conversations.

Traditional chatbots often fail because they:

- Lack memory.
- Cannot access enterprise knowledge bases.
- Cannot retrieve current information from the web.
- Execute all tools unnecessarily, increasing latency and cost.

This solution addresses these challenges through an intelligent agent architecture.

3. Solution Objectives

The chatbot should support three operational modes:

Mode 1: General Chat

The agent answers questions using only the Large Language Model.

Examples:

- Explain Artificial Intelligence.
- What is Machine Learning?
- Write a Python function to sort a list.

No external tools should be invoked.

Mode 2: Web Search Mode

The agent retrieves real-time information from the internet when required.

Examples:

- Latest AI news.
- Current stock market updates.
- Recent technology trends.

The agent must invoke a Web Search Tool and use retrieved results to formulate a response.

Mode 3: RAG Mode

The agent retrieves information from enterprise documents stored in a vector database.

Examples:

- What is the company's leave policy?
- Summarize the uploaded project proposal.
- Explain the onboarding process mentioned in the HR handbook.

The agent must retrieve relevant chunks from the vector index and answer based on retrieved context.

4. Core Functional Requirements

Multi-Agent Chat Experience

The solution must expose a single chat interface where users can interact naturally.

The system should support:

- Natural language conversations
- Multi-turn interactions
- Context retention
- Source-aware responses

Intelligent Tool Selection

The agent must not invoke all tools for every query.

Instead, it should determine:

General Query

Use:

- LLM only

Real-Time Query

Use:

- Web Search Tool

Enterprise Knowledge Query

Use:

- RAG Retrieval Tool

This decision-making should be performed dynamically by the agent.

Conversational Memory

The system must retain previous interactions.

Capabilities:

- Follow-up questions
- Context-aware responses
- Session continuity

Example:

User: "What is LangGraph?"

Agent: "LangGraph is a framework for building stateful AI workflows."

User: "Who created it?"

Agent: "It was developed by LangChain."

Without memory, the agent would not understand the reference to "it."

5. System Components

5.1 Streamlit Frontend

Responsibilities:

- Chat interface
- Conversation history
- Mode display
- Source display
- User session management

Features:

- Clean conversational UI
- Streaming responses
- Source citations
- Chat history panel

5.2 Agent Orchestrator

Responsibilities:

- Receive user queries
- Determine execution path
- Invoke required tools
- Maintain state
- Generate final response

Possible Frameworks:

- LangGraph
- CrewAI

- Haystack Agents

Preferred: LangGraph

Reason: Provides stateful workflows and tool orchestration.

5.3 Memory Component

Responsibilities:

- Store conversation history
- Retrieve relevant past messages
- Enable contextual responses

Memory Types:

Short-Term Memory

Stores current session messages.

Examples:

- Chat history
- Session state

Long-Term Memory

Stores important user interactions.

Examples:

- User preferences
- Frequently discussed topics

Storage Options:

- SQLite
- PostgreSQL
- ChromaDB Metadata

Preferred: SQLite for hackathon simplicity.

5.4 Web Search Tool

Purpose:

Retrieve real-time information from the internet.

Responsibilities:

- Search web
- Retrieve top results
- Summarize findings

Possible Open-Source Options:

- Tavily Search
- DuckDuckGo Search
- SerpAPI (optional)

Preferred: DuckDuckGo Search

Reason: Completely free and open source.

Workflow:

User Query → Search Tool → Top Results → LLM Summary → Response

.

5.5 RAG Retrieval Tool

Purpose:

Retrieve enterprise knowledge.

Responsibilities:

- Query vector database
- Fetch relevant chunks
- Return contextual information

Workflow:

User Query → Embedding Model → Vector Search → Top K Documents → LLM → Response

.

5.6 Vector Database

Stores document embeddings.

Supported Options:

- ChromaDB
- FAISS
- Qdrant
- Weaviate

Preferred: ChromaDB

Reason: Simple setup and fully open source.

5.7 Embedding Model

Converts text into vectors.

Options:

- BGE Small
- BGE Base
- E5 Small
- Instructor XL

Preferred: BAAI/bge-small-en-v1.5

Reason: Excellent performance with low resource requirements.

5.8 Large Language Model

Responsible for:

- Reasoning
- Tool selection
- Answer generation

Open Source Models:

- Llama 3
- Mistral
- Gemma
- Qwen

Preferred: Llama 3 8B Instruct

Deployment Options:

- Ollama
- vLLM
- HuggingFace Transformers

Preferred: Ollama

Reason: Simple local deployment.

6. Agent Decision Flow

Step 1: Receive user question

Step 2: Analyze intent

Step 3: Determine action

Decision Matrix:

Query Type	Action
General Knowledge	LLM
Current Events	Web Search Tool
Enterprise Documents	RAG Tool
Follow-up Question	Memory + Appropriate Tool

Step 4: Generate response

Step 5: Store conversation in memory

7. Non-Functional Requirements

Performance

- Response time < 5 seconds
- Vector retrieval < 1 second
- Web search < 3 seconds

Scalability

- Multiple users
- Concurrent sessions

Reliability

- Tool failure handling
- Graceful fallbacks

Security

- Secure document storage
- Session isolation

8. Recommended Open-Source Technology Stack

Layer	Technology
UI	Streamlit
Agent Framework	LangGraph
LLM	Groq Models
Embeddings	Hugging Face Models
Vector DB	FAISS
RAG Framework	LangChain
Backend	Python
Document Parsing	Unstructured

9. Evaluation Criteria

Teams will be evaluated on:

Functional Completeness

- All three modes implemented

Intelligent Tool Routing

- Correct tool selection
- No unnecessary tool invocation

Memory Handling

- Accurate follow-up responses

RAG Accuracy

- Relevant document retrieval

User Experience

- Streamlit interface quality

Architecture

- Modularity
- Scalability
- Maintainability

10. Expected Deliverables

Each team must submit:

1. Source Code Repository
2. Streamlit Application
3. Documents used for RAG
4. Presentation Deck (Optional)

11. Expected Outcome

The final solution should demonstrate an enterprise-grade conversational AI assistant capable of:

- General question answering
- Real-time web search
- Enterprise document retrieval
- Context-aware follow-up conversations
- Intelligent tool orchestration

The chatbot should act as a smart AI agent that uses tools only when required, thereby improving response quality, reducing latency, and providing a natural user experience.